template_main.typ

one Typst template for notes, homework, papers and books

Reference · September 2026

1 **Getting started** **1**
1.1 Installing it 2
1.2 What is in the directory 2
1.3 Using it with Claude Code 3
1.4 The four shells 3
1.5 Options 4
1.6 A masthead of your own 5
2 **Environments** **5**
2.1 What is available 6
2.2 Renaming and restyling one box 7
2.3 Boxes with a name instead of a number 8
2.4 Rolling your own 8
3 **Writing mathematics** **9**
3.1 Shorthands 9
3.2 Equation numbering 9
4 **Structure** **9**
4.1 Problem headings 9
4.2 Appendices 10
4.3 Contents 10
5 **Figures** **10**
5.1 Function plots 11
6 **Design** **12**
6.1 Tokens 12
6.2 Fonts 12
6.3 Other pieces 13
7 **Recipes** **13**
7.1 A lecture note 13
7.2 A problem set 14
7.3 A paper 14
7.4 A book 14
8 **Moving old documents over** **15**

1 Getting started

This is a Typst **local package**, so a document anywhere on the machine reaches it in one line and there are no relative paths to keep straight. Every document begins the same two lines:

```
#import "@local/folio:0.1.0": *
#show: notes.with(title: "Eigenvalues", course: "MATH 221")
```

Then compile:

```
typst compile notes.typ          # once, to notes.pdf
typst watch notes.typ            # recompile on every save
typst compile notes.typ out.png  # or PNG, SVG, HTML
```

To start from a working skeleton rather than a blank page:

```
typst init @local/folio my-notes
```

Nothing else needs installing: the template is plain Typst, and figures are drawn natively, so a finished document is one `.typ` file. A relative import — `#import "../templates/template_main.typ": *` — still works everywhere the package form does, if you would rather not install anything at all.

1.1 Installing it

```
./install.sh           # link it — edits apply immediately (the default)
./install.sh --copy    # copy it — a frozen snapshot
./install.sh --force   # replace an install already there
./install.sh --skill   # also install the Claude Code skill
```

The script reads `name`, `version` and `entrypoint` out of `typst.toml` and puts them where Typst looks for local packages:

PLATFORM	WHERE IT LANDS
Linux	`~/.local/share/typst/packages/local/folio/0.1.0`
macOS	`~/Library/Application Support/typst/packages/local/folio/0.1.0`
Windows	`%APPDATA%\typst\packages\local\folio\0.1.0`

The default links rather than copies, so the installed package *is* this directory: edit `template_main.typ` and the next compile of any document anywhere picks the change up. Use `--copy` when you would rather have documents pinned to a snapshot that later edits cannot disturb.

Renaming it. Change `name` in `typst.toml` and run `./install.sh` again. Nothing inside the template refers to the package name — only `template/main.typ`, the `typst init` skeleton, which imports it.

Versioning it. A version is just a directory, so `0.1.0` and `0.2.0` sit side by side and old documents go on compiling against the version they named. Bump `version` in `typst.toml`, run `./install.sh`, and both are live.

1.2 What is in the directory

FILE	WHAT IT IS
`template_main.typ`	the template — the package entrypoint

`typst.toml`	the package manifest: name, version, entrypoint
`install.sh`	puts it where Typst looks
`template/main.typ`	the skeleton `typst init` copies
`skills/folio/SKILL.md`	a Claude Code skill for writing with it
`archive/`	the four documents this was distilled from
`README.typ`	this file

1.3 Using it with Claude Code

`skills/folio/SKILL.md` teaches Claude Code the template: which shell to pick, every environment and its overrides, the math shorthands, the figure primitives, and the handful of Typst gotchas that cost real debugging time.

```
./install.sh --skill     # user level: available in every project
```

That links it to `~/.claude/skills/folio`. For one project only, link it there instead:

```
mkdir -p .claude/skills
ln -s /path/to/typstTemplates/skills/folio .claude/skills/folio
```

Because the install is a link, editing `SKILL.md` updates what Claude reads — keep it in step with the template when you change either.

1.4 The four shells

The shell is the `#show:` line. All four share one design language and one set of environments; they differ only in what the top of the document looks like and how things are numbered.

SHELL	FRONT MATTER	SECTIONS	EQUATIONS
`notes`	a masthead: title, then author · course · date	`1.1`	by section
`homework`	the same masthead	`1.` `(a)` `(i)`	unnumbered
`paper`	centred title block, optional abstract	`1.1`	`(1)`, flat
`book`	title page and contents page	chapter openers	by chapter

Pick the one that matches the document and pass it what it needs:

```
#show: notes.with(title: "Spectral theorem", course: "MATH 221",
                  author: "Aren Vista", date: "2026-09-12")

#show: homework.with(title: "Problem Set 4", course: "MATH 302",
                     author: "Aren Vista", count-h1: true)

#show: paper.with(title: "On the Spectral Gap", author: "Aren Vista",
                  date: "September 2026",
                  abstract: [We show that the gap is bounded below.])
```

```
#show: book.with(title: "Differential Equations", course: "MATH 225",
                 subtitle: "a first course", author: "Aren Vista",
                 date: "Fall 2025",
                 blurb: [Lecture notes and worked examples, collected.])
```

1.5 Options

Every shell accepts every option below; they differ only in the defaults. Anything you pass wins over the default, so `#show: notes.with(chapters: true)` is a perfectly good thing to write.

OPTION	DEFAULT	WHAT IT DOES
`title`	`none`	the document title
`subtitle`	`none`	a line under it
`author`	`none`	byline; also the PDF metadata author
`course`	`none`	the eyebrow above the title
`date`	`none`	shown in the byline
`blurb`	`none`	a paragraph on a title page (`book`)
`abstract`	`none`	set narrow under the title (`paper`)
`paper-size`	`"us-letter"`	any Typst paper name, e.g. `"a4"`
`margin`	`auto`	`auto` picks per shell; or a dictionary
`font`	`font-serif`	body font stack
`size`	`11pt`	body size
`page-fill`	`white`	try `ground` for a tinted page
`justify`	`true`	justified body text
`numbered`	`true`	show numbers on the environment boxes
`eq-numbering`	`"section"`	`"section"` , `"flat"` , `none` , or a pattern
`head-numbering`	`"1.1"`	any Typst numbering pattern or function
`cover`	per shell	`"banner"` , `"center"` , `"page"` , `none` — or your own; see below
`cover-rule`	`"gradient"`	`"gradient"` , `"hairline"` , or `none`
`toc`	`false`	insert a contents listing
`chapters`	`false`	level-1 headings open a new page

`homework` adds two of its own: `count-h1: true` makes a level-1 heading read “Question 2” rather than “2. Question” (write `= Question` and let the counter supply the number), and `h1-label` replaces the heading text entirely, so every level-1 heading reads “Problem 1”, “Problem 2”, and so on.

1.6 A masthead of your own

Besides the four built-in names, `cover` takes a **function** — called with everything the document was given — or ready-made **content**, used as it stands.

KEY	WHAT IT HOLDS
`title` , `subtitle`	as passed to the shell
`author` , `course` , `date`	as passed to the shell
`blurb` , `abstract`	as passed to the shell
`rule`	the bar `cover-rule` selects, ready to place

```
#show: notes.with(
  title: "Orthogonality", course: "MATH 221",
  author: "Aren Vista", date: "2026-09-12",
  cover: m => {
    grid(
      columns: (1fr, auto),
      align: (left + bottom, right + bottom),
      {
        eyebrow(m.course)
        v(0.3em)
        text(size: 22pt, weight: 600, fill: ink, m.title)
      },
      text(size: 9.5pt, fill: muted)[#m.author \ #m.date],
    )
    m.rule
  },
)
```

Any key may be `none` — the shell passes on whatever it was handed — so a cover that reads `m.date` should cope with a document that has no date. `m.rule` is content rather than a function, because Typst will not call a dictionary key directly; use `hrule(above: .., below: ..)` when you want the bar at a spacing of your own.

For a masthead that does not vary with the document, hand `cover` the content itself:

```
#show: notes.with(cover: [
  #align(center, text(size: 24pt, weight: 600)[Seminar notes])
  #v(1em)
])
```

And `cover: none` leaves the top of the page bare, for a document that opens straight into its first heading.

2 Environments

A box is one function call. The title is optional; the body is the content block that follows.

```
#definition(title: "Eigenvector")[
  A nonzero $bold(v)$ with $A bold(v) = lambda bold(v)$.
]
```

```
#theorem[Every operator on $CC^n$ has an eigenvalue.]
#proof[Consider $I, T, T^2, dots, T^n$.]
```

Definition (Eigenvector) A nonzero v with $Av = \lambda v$.

Theorem Every operator on $\mathbb{C}^n$ has an eigenvalue.

Proof Consider $I, T, T^2, \ldots, T^n$ and take a linear dependence. ∎

2.1 What is available

Two axes of information: the hue names the family, the glyph names the environment. A reader can tell a proved result from a definition from an aside without reading the label.

Results are asserted and proved.

Definitions fix language, assert nothing.

Keepsakes are worth having to hand.

Instances are concrete.

Our work is our own reasoning.

Prompts are what was asked of us.

Asides are commentary and scaffolding.

Unsure is stated but not established.

Alerts flag a trap worth avoiding.

FUNCTION	ALIAS	NUMBERED	FAMILY
`theorem`	`thm`	yes	result
`proposition`	`prop`	yes	result
`lemma`	`lem`	yes	result
`corollary`	`cor`	yes	result
`claim`		yes	result
`definition`	`defn` , `def`	yes	definition
`axiom`		yes	definition
`notation`		yes	definition
`formula`		yes	keepsake
`key`		no	keepsake
`example`	`exm`	yes	instance
`exercise`		yes	prompt
`conjecture`	`conj`	yes	unsure

`proof`	`pf`	no	our work
`solution`	`soln`	no	our work
`intuition`		no	our work
`question`	`ques`	no	prompt
`remark`		no	aside
`aside`		no	aside
`observation`		no	aside
`case`		no	aside
`warning`	`warn`	no	alert
`important`	`imp`	no	alert

`proof` closes itself with a QED tombstone. Numbering is shared: one counter serves every box, so "Theorem 3.7" is the seventh numbered box in section 3 whatever kind it is, and the counter restarts at each level-1 heading. Boxes marked "no" above are never numbered — a proof belongs to the thing above it, and an aside is not a result.

Turn numbering off entirely with `numbered: false` on the shell; the labels then read "Theorem" with no number.

2.2 Renaming and restyling one box

Every environment takes `kind`, `glyph`, `hue` and `counted` as named arguments. The defaults are what the table above lists; override any of them at the point of use.

WRITTEN	GIVES
`#question[...]`	the default: `?` glyph, "Question" label
`#question(kind: "Prompt")[...]`	the same box, relabelled
`#question(glyph: none)[...]`	no glyph; the label starts flush left
`#question(kind: none)[...]`	no label at all — just the body
`#question(kind: none, title: "Warm-up")[...]`	the title becomes the label
`#question(hue: c-iter)[...]`	another hue for this one box
`#theorem(counted: false)[...]`	numbered normally, but not this one

? **Prompt** Renamed: the intro word is whatever you pass.

Question No glyph — the label starts at the left edge.

Neither label nor glyph: a plain tinted box.

? **Warm-up** A title on its own becomes the label.

To change one for a whole document rather than one box, rebind it once near the top — `.with` fills in the new defaults and everything else stays put:

```
#let question = question.with(kind: "Prompt", glyph: [*])
#let theorem = theorem.with(kind: "Satz")
#let proof = proof.with(kind: "Beweis", glyph: [□])
```

One wrinkle worth knowing: `kind: none` hides the number along with the label, but the box still **consumes** one, so the next numbered box carries on from where it would have. Pass `counted: false` as well if it should not count at all.

2.3 Boxes with a name instead of a number

For commentary that wants a label rather than a number. These carry the rotated square the lists use, so they read as a quieter family.

```
#pitfall[Rows before columns — always, and mixing them up is the single
         most common avoidable error in this course.]
#idea[Elimination never changes the solution set, only the presentation.]
#recall[The rank-nullity theorem, from Chapter 4.]
#notebox("Worth remembering", hue: c-iter)[Any label you like.]
```

◆ COMMON PITFALL Rows before columns — always, and mixing them up is the single most common avoidable error in this course.

◆ WORTH REMEMBERING Any label you like, in any hue.

2.4 Rolling your own

`callout` is what all of the above are built from:

```
#let heuristic(title: none, body) = callout(
  "Heuristic", body, hue: c-iter, glyph: [↯], title: title, counted: false,
)
#heuristic[When in doubt, diagonalise.]
```

↯ **Heuristic** When in doubt, diagonalise.

Its arguments are `kind` and `body` positionally, then `hue`, `glyph`, `title`, `counted`, `italic` and `breakable`. `kind: none` and `glyph: none` work here too, so `callout` covers anything the named environments do not.

3 Writing mathematics

3.1 Shorthands

WRITTEN	GIVES	NOTES
`$RR$` , `$NN$` , `$ZZ$`	$\mathbb{R}, \mathbb{N}, \mathbb{Z}$	also `QQ CC FF PP EE HH KK`
`$ip(u, v)$`	$\langle u, v \rangle$	inner product
`$norm(x)$`	$\|x\|$	norm
`$abs(x)$` , `$card(S)$`	$\lvert x \rvert$, $\lvert S \rvert$	bars
`$setb(x in RR, x > 0)$`	$\{x \in \mathbb{R} : x > 0\}$	set-builder
`$dv(y, x)$`	$\frac{\mathrm{d}y}{\mathrm{d}x}$	derivative
`$pdv(f, x)$`	$\frac{\partial f}{\partial x}$	partial derivative
`$restr(f, A)$`	$f\|_A$	restriction
`$evalat(F(x), a, b)$`	$F(x)\|_a^b$	evaluation bar
`$vv(1, 2, 3)$`	$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$	bracketed column
`$mm(1, 2; 3, 4)$`	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	bracketed matrix, `;` between rows
`$eps$` , `$dt$`	ε, $\cdot$	epsilon, dot

Operators Typst does not ship are defined here: `rank` , `span` , `Span` , `Nul` , `Col` , `Row` , `sgn` , `diag` , `adj` , `nullity` , `proj` , `comp` , `argmin` , `argmax` , `grad` , `curl` , `divg` , `st` , `Aut` , `End` , `Hom` , `GL` , `SL` . Typst already has `det` , `dim` , `ker` , `tr` , `im` , `id` , `hom` , `Pr` , `gcd` , `lcm` , `min` , `max` , `inf` , `sup` and `lim` — do not redefine those.

$$\operatorname{rank} A + \operatorname{nullity} A = \dim \mathbb{R}^n, \quad \operatorname{proj}_{\boldsymbol{u}} \boldsymbol{v} = \frac{\langle \boldsymbol{v}, \boldsymbol{u} \rangle}{\langle \boldsymbol{u}, \boldsymbol{u} \rangle} \boldsymbol{u} \tag{3.1}$$

3.2 Equation numbering

`eq-numbering: "section"` numbers equations `(1.1)` , `(1.2)` , restarting at each level-1 heading; `"flat"` counts straight through as `(1)` , `(2)` ; and `none` turns numbering off, which is what `homework` does. Refer to one by labelling it:

```
$ e^(i pi) + 1 = 0 $ <euler>
As @euler shows, ...
```

4 Structure

4.1 Problem headings

`#problem(n)` writes an unnumbered level-1 heading that still lands in the outline and still resets the environment counter.

```
#problem(1)[ ... ]
#problem(2, title: "Cauchy sequences", newpage: true)[ ... ]
```

4.2 Appendices

Everything after `#show: appendix` is lettered — "Appendix A", section A.2 — and the boxes inside number as A.3 rather than 11.3.

```
#show: appendix
= Quick Reference
```

4.3 Contents

`toc: true` on the shell inserts one. Call `#contents()` yourself to put it somewhere specific, and `#front-title[Preface]` for a front-matter heading that takes the chapter opener's look without stepping the chapter counter or entering the outline.

```
#front-title[Preface]
...
#pagebreak()
#front-title[Contents]
#contents(depth: 3)
```

5 Figures

Figures are drawn natively: no packages, no image files, and they scale and re-colour with the rest of the document. Coordinates are plain numbers in points, x rightward and y downward from the top-left corner of the canvas.

```
#figframe[Orthogonal projection of $bold(v)$ onto the line through $bold(u)$.][
  #fcanvas(300, 130, {
    fseg((10, 110), (290, 110), col: rule-line)
    fvec((40, 110), (250, 40), col: c-fig)     // v
    fvec((40, 110), (200, 110), col: c-fig2)   // proj
    fseg((250, 40), (250, 110), dash: "dashed", col: muted)
    fdot((40, 110), col: ink)
    flab((255, 30), $bold(v)$, col: c-fig)
    flab((120, 125), $proj bold(v)$, col: c-fig2)
    frt((250, 110), (-1, 0), (0, -1))
  })
]
```

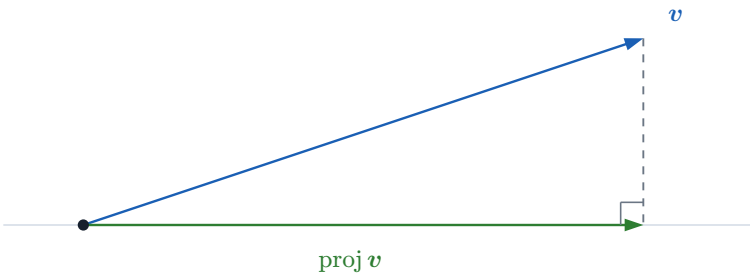
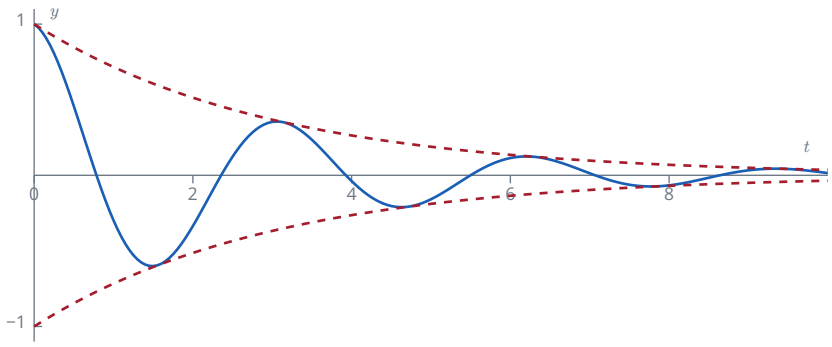


FIGURE 5.1 Orthogonal projection of v onto a line.

HELPER	DRAWS
`fcanvas(w, h, body)`	the drawing surface, `w` and `h` in points
`figframe(caption, body)`	a numbered, captioned frame
`fseg(p, q)`	a line segment; `col`, `th`, `dash`
`fvec(p, q)`	an arrow from `p` to `q`
`fdot(p)`	a filled dot; `col`, `r`
`flab(p, body)`	a label centred on `p`
`fpoly(pts)`	a filled polygon; `fill`, `stroke`
`frt(c, d1, d2)`	a right-angle tick at the corner `c`
`fhead(x, y, a)`	a bare arrowhead at angle `a`

5.1 Function plots

```
#plot(
  (
    (fn: x => calc.exp(-x / 3) * calc.cos(2 * x), paint: c-core),
    (fn: x => calc.exp(-x / 3), paint: c-solve, dash: "dashed"),
    (fn: x => -calc.exp(-x / 3), paint: c-solve, dash: "dashed"),
  ),
  xlim: (0, 10), ylim: (-1.1, 1.1),
  xticks: (0, 2, 4, 6, 8), yticks: (-1, 1),
  xlabel: $t$, ylabel: $y$,
)
```



Each segment is a dictionary: `fn` is required, and `from`, `to`, `paint` and `dash` are optional. `marks` places dots — `(x: 2, y: 0.5)`, with `open: true` for a hollow one — which is how you draw an open endpoint on a piecewise function.

6 Design

6.1 Tokens

Change these at the top of `template_main.typ` and the whole system follows: the boxes, rules, tables and figures all read from them.

TOKEN		ROLE
`ink`		body text
`ink-soft`		secondary text, table cells
`muted`		eyebrows, captions
`rule-line`		hairlines that separate
`rule-soft`		hairlines that only hint
`ground`		an optional tinted page
`c-core`		blue — the core thread; also `accent`
`c-ops`		green — operations
`c-decomp`		purple — decompositions
`c-solve`		red — solving
`c-iter`		olive — iteration

The environment hues are `c-result`, `c-define`, `c-argue`, `c-illus`, `c-prompt`, `c-aside`, `c-unsure`, `c-alert` and `c-formula`. All of them clear WCAG AA against their own tinted fill, so a label stays legible.

6.2 Fonts

`font-serif`, `font-sans` and `font-mono` are the stacks; `font-cm` is there if you want Computer Modern for the body. They are kept short on purpose — Typst warns once for every family it cannot find — so put the faces you actually own at the front:

```
#import "@local/folio:0.1.0": *
#let my-serif = ("Iowan Old Style", "Charter", ..font-serif)
#show: notes.with(title: "...", font: my-serif)
```

6.3 Other pieces

The gradient bar is the loudest thing in the design language, so it appears only where a document opens: under the masthead of a note or a problem set, under the title block of a paper, on a book's title page, and at the head of every chapter. Pass `cover-rule: "hairline"` for a quiet grey line instead, or `cover-rule: none` for bare space. Chapter openers always carry the bar — it is what marks a chapter.

HELPER	WHAT IT MAKES
`hrule()`	the gradient bar; `w` , `above` , `below`
`thinrule()`	a plain hairline
`eyebrow[Chapter]`	sans, uppercase, letterspaced, muted
`hnum[3.1]`	the mono numeral that sits before a heading
`front-title[Preface]`	a front-matter title
`legend(kind, hue, glyph, desc)`	one row of a colour key
`sqmark(hue)`	the rotated square the lists use

Tables, code blocks, block quotes and lists are styled by the shell; write them the ordinary Typst way and they come out in the same system.

> A block quote takes an accent border and the soft ink.

7 Recipes

7.1 A lecture note

```
#import "@local/folio:0.1.0": *
#show: notes.with(
  title: "Orthogonality",
  course: "MATH 221",
  author: "Aren Vista",
  date: "2026-09-12",
)

= Inner products
#definition(title: "Inner product")[ ... ]
#theorem[Cauchy–Schwarz.] #proof[ ... ]

== Orthogonal projection
#example[ ... ]
#pitfall[ ... ]
```

7.2 A problem set

```
#import "@local/folio:0.1.0": *
#show: homework.with(
  title: "Problem Set 4",
  course: "MATH 302",
  author: "Aren Vista",
  count-h1: true,
)

= Question
Show that every Cauchy sequence in $RR$ converges.
#solution[ ... ]

== Part
#solution[ ... ]
```

Or, if you would rather number the problems yourself:

```
#problem(1, title: "Cauchy sequences")[
  #claim[Every Cauchy sequence is bounded.]
  #proof[ ... ]
]
```

7.3 A paper

```
#import "@local/folio:0.1.0": *
#show: paper.with(
  title: "On the Spectral Gap",
  author: "Aren Vista",
  date: "September 2026",
  abstract: [We show that the gap is bounded below by ... ],
)

= Introduction
...
#bibliography("refs.bib")
```

7.4 A book

```
#import "@local/folio:0.1.0": *
#show: book.with(
  title: "Differential Equations",
  subtitle: "a first course",
  course: "MATH 225",
  author: "Aren Vista",
  date: "Fall 2025",
  blurb: [Lecture notes, worked examples and problem sets, collected.],
)

#front-title[Preface]
...
```

```
= First Notions          // a chapter: opens a page under the gradient rule
== What is a differential equation?
#definition(title: "Differential equation")[ ... ]

#show: appendix
= Quick Reference
```

8 Moving old documents over

The four files this template was assembled from now live in `archive/`; `textbook.typ` and `LinearAlgebra.typ` carry their own preambles and still compile unchanged. Names map across like this.

WAS	IS NOW	NOTE
`notes`, `homework`	unchanged	`numbered` now defaults to `true`
`problem`	unchanged	same arguments
`textbook(...)`	`book(...)`	`term:` is now `date:`
`hr()`	`hrule()`	takes `above` and `below`
`hum`	`intuition`	
`form`	`formula`	
`imp`	`important`	`imp` kept as an alias
`ques`	`question`	`ques` kept as an alias
`soln`	`solution`	`soln` kept as an alias
`defn`, `def`, `thm`, `exm`	unchanged	aliases of the long names
`aside(col, label, body)`	`notebox(label, body, hue: col)`	`aside` is now an environment
`calloutbox`	`notebox`	
`_labeled`, `_callout`	`callout`	now public, for your own boxes
`lin-ink`, `ink`	`ink`	the whole `lin-*` palette became tokens
`lin-core` … `lin-iter`	`c-core` … `c-iter`	
`cDefn`, `cThm`, `cEx`, `cWarn`, `cNote`	`c-define`, `c-result`, `c-illus`, `c-alert`, `c-aside`	
`fSerif`, `fSans`, `fMono`	`font-serif`, `font-sans`, `font-mono`	
`dm`, `tr`	`dim`, `tr`	Typst ships both; the aliases are gone
`coord`	unchanged	`ip` is the inner product, `coord` the pair

The figure helpers (`fcanvas`, `figframe`, `fseg`, `fvec`, `fdot`, `flab`, `fpoly`, `frt`, `fhead`), `plot`, `eyebrow`, `hnum`, `front-title`, `legend`, `pitfall`, `idea` and `recall` all kept their names and their arguments.